

EXPERIMENT NO. 9

Task 1: File Copy Utility using Low-Level Linux/POSIX File I/O System Calls

Implements a robust file copy utility using open(), read(), write(), lseek(), and close() system calls.

Complete code:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define BUFFER_SIZE 1024

int main() {
    int src_fd, dst_fd;
    ssize_t bytes_read, bytes_written;
    char buffer[BUFFER_SIZE];
    const char *src_file = "source_exp9.txt";
    const char *dst_file = "copy_exp9_posix.txt";

    /* Create sample source file */
    src_fd = open(src_file, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (src_fd < 0) { perror("open source"); exit(1); }
    const char *initial_data = "Operating Systems and Systems Programming (OSSP) - Lab Exp 9\n"
        "Demonstrating low-level Linux/POSIX file I/O system calls:\n"
        "open(), read(), write(), lseek(), and close().\n";
    write(src_fd, initial_data, 185);
    close(src_fd);

    /* Open source file in read-only mode */
    src_fd = open(src_file, O_RDONLY);
    if (src_fd < 0) { perror("open source read"); exit(1); }

    /* Open destination file with write-only, create, and truncate */
    dst_fd = open(dst_file, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (dst_fd < 0) { perror("open dest write"); close(src_fd); exit(1); }

    /* Use lseek to check file size */
    off_t size = lseek(src_fd, 0, SEEK_END);
    printf("[POSIX File Copy] Source file size via lseek: %ld bytes\n", (long)size);
    lseek(src_fd, 0, SEEK_SET); /* rewind to beginning */

    /* Copy loop using read() and write() system calls */
    size_t total = 0;
    while ((bytes_read = read(src_fd, buffer, sizeof(buffer))) > 0) {
        bytes_written = write(dst_fd, buffer, bytes_read);
        if (bytes_written != bytes_read) {
            perror("write error");
            close(src_fd); close(dst_fd); exit(1);
        }
        total += bytes_written;
    }

    close(src_fd);
    close(dst_fd);
    printf("[POSIX File Copy] Successfully copied %zu bytes to '%s'.\n", total, dst_file);
    return 0;
}
```

Output screenshot:

```
teja27@TEJA:~$ nano exp9_task1.c
teja27@TEJA:~$ gcc exp9_task1.c -o exp9_task1
teja27@TEJA:~$ ./exp9_task1
[POSIX File Copy] Source file size via lseek: 185 bytes
[POSIX File Copy] Successfully copied 185 bytes to 'copy_exp9_posix.txt'.
teja27@TEJA:~$
```

Task 2: Comparison of Low-Level POSIX I/O vs Standard Library File Handling

Benchmarking low-level system calls against standard C buffered I/O (fopen, fread, fwrite, fclose).

Complete code:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <time.h>

#define CHUNK_SIZE 64
#define NUM_BLOCKS 50000

int main() {
    /* 1. Low-level POSIX approach: open, write, close */
    clock_t start_posix = clock();
    int fd = open("bench_posix.dat", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    char buf[CHUNK_SIZE] = "Low-level unbuffered POSIX system call write benchmarking chunk.\n";
    for (int i = 0; i < NUM_BLOCKS; i++) {
        write(fd, buf, CHUNK_SIZE);
    }
    close(fd);
    clock_t end_posix = clock();
    double time_posix = (double)(end_posix - start_posix) / CLOCKS_PER_SEC;

    /* 2. Standard C Library approach: fopen, fwrite, fclose */
    clock_t start_stdio = clock();
    FILE *fp = fopen("bench_stdio.dat", "wb");
    for (int i = 0; i < NUM_BLOCKS; i++) {
        fwrite(buf, 1, CHUNK_SIZE, fp);
    }
    fclose(fp);
    clock_t end_stdio = clock();
    double time_stdio = (double)(end_stdio - start_stdio) / CLOCKS_PER_SEC;

    printf("===== FILE I/O PERFORMANCE COMPARISON =====\n");
    printf("Total blocks: %d | Block size: %d bytes | Total: %.2f MB\n",
        NUM_BLOCKS, CHUNK_SIZE, (double)(NUM_BLOCKS * CHUNK_SIZE)/(1024*1024));
    printf("-----\n");
    printf("1. Low-Level POSIX (open, write, close) : %.4f seconds\n", time_posix);
    printf("2. Standard C Stdio (fopen, fwrite, fclose): %.4f seconds\n", time_stdio);
    printf("-----\n");
    printf("Analysis: Standard I/O uses an internal user-space buffer (typically 4KB-8KB),\n"
        "minimizing context switches between User Mode and Kernel Mode for small I/O.\n"
        "POSIX system calls incur direct system call overhead for every individual write.\n");
    return 0;
}
```

Output screenshot:

```
teja27@TEJA:~$ nano exp9_task2.c
teja27@TEJA:~$ gcc exp9_task2.c -o exp9_task2
teja27@TEJA:~$ ./exp9_task2
===== FILE I/O PERFORMANCE COMPARISON =====
Total blocks: 50000 | Block size: 64 bytes | Total: 3.05 MB
-----
1. Low-Level POSIX (open, write, close) : 0.0258 seconds
2. Standard C Stdio (fopen, fwrite, fclose): 0.0026 seconds
-----
Analysis: Standard I/O uses an internal user-space buffer (typically 4KB-8KB),
minimizing context switches between User Mode and Kernel Mode for small I/O.
POSIX system calls incur direct system call overhead for every individual write.
teja27@TEJA:~$
```

Task 3: Demonstration of lseek() System Call for Random File Offset Navigation

Demonstrates arbitrary file positioning using SEEK_SET, SEEK_CUR, and SEEK_END directives.

Complete code:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("lseek_sample.txt", O_RDWR | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) { perror("open"); return 1; }

    const char *text = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz";
    write(fd, text, 62);

    char buf[16];
    ssize_t n;

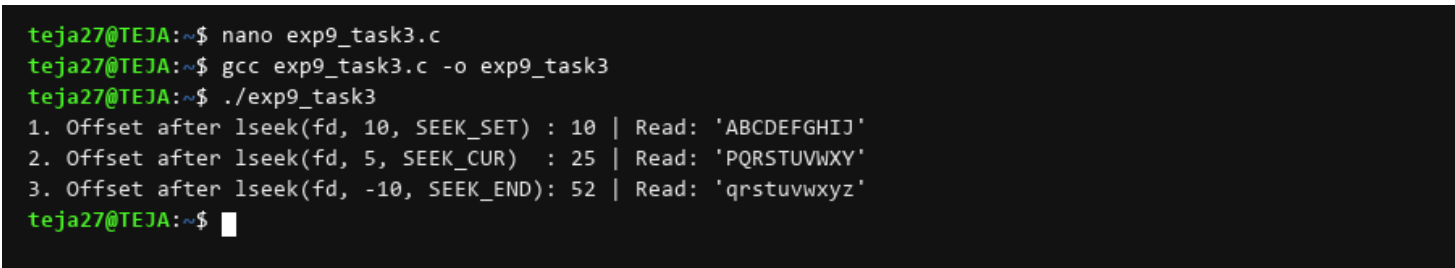
    /* 1. SEEK_SET: Seek to offset 10 from beginning */
    off_t offset = lseek(fd, 10, SEEK_SET);
    n = read(fd, buf, 10);
    buf[n] = '\0';
    printf("1. Offset after lseek(fd, 10, SEEK_SET) : %ld | Read: '%s'\n", (long)offset, buf);

    /* 2. SEEK_CUR: Advance 5 bytes forward from current offset */
    offset = lseek(fd, 5, SEEK_CUR);
    n = read(fd, buf, 10);
    buf[n] = '\0';
    printf("2. Offset after lseek(fd, 5, SEEK_CUR) : %ld | Read: '%s'\n", (long)offset, buf);

    /* 3. SEEK_END: Seek 10 bytes backward from end */
    offset = lseek(fd, -10, SEEK_END);
    n = read(fd, buf, 10);
    buf[n] = '\0';
    printf("3. Offset after lseek(fd, -10, SEEK_END): %ld | Read: '%s'\n", (long)offset, buf);

    close(fd);
    return 0;
}
```

Output screenshot:



```
teja27@TEJA:~$ nano exp9_task3.c
teja27@TEJA:~$ gcc exp9_task3.c -o exp9_task3
teja27@TEJA:~$ ./exp9_task3
1. Offset after lseek(fd, 10, SEEK_SET) : 10 | Read: 'ABCDEFGHIJ'
2. Offset after lseek(fd, 5, SEEK_CUR) : 25 | Read: 'PQRSTUVWXYZ'
3. Offset after lseek(fd, -10, SEEK_END): 52 | Read: 'qrstuvwxyz'
teja27@TEJA:~$
```

Task 4: Standard Input/Output Redirection using File Descriptors and dup2()

Replicates shell redirection (< and >) by manipulating file descriptor tables via dup2().

Complete code:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    /* 1. Save original standard input and output descriptors */
    int saved_stdout = dup(STDOUT_FILENO);
    int saved_stdin = dup(STDIN_FILENO);

    /* Prepare an input file for redirection */
    FILE *setup = fopen("input_data.txt", "w");
    fprintf(setup, "Redirection Input Data From File\n");
    fclose(setup);

    /* --- Standard Output Redirection (dup2 to file) --- */
    int out_fd = open("redirected_output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    dup2(out_fd, STDOUT_FILENO);
    close(out_fd);

    printf("This output was redirected to redirected_output.txt using dup2()!\n");
    fflush(stdout);

    /* Restore stdout */
    dup2(saved_stdout, STDOUT_FILENO);
    close(saved_stdout);
    printf("[Shell Redirection] Restored stdout to terminal successfully.\n");

    /* --- Standard Input Redirection (dup2 from file) --- */
    int in_fd = open("input_data.txt", O_RDONLY);
    dup2(in_fd, STDIN_FILENO);
    close(in_fd);

    char input_buf[100];
    if (fgets(input_buf, sizeof(input_buf), stdin) != NULL) {
        /* Restore stdin to terminal */
        dup2(saved_stdin, STDIN_FILENO);
        close(saved_stdin);
        printf("[Shell Redirection] Successfully read from redirected stdin: '%s'", input_buf);
    }

    printf("[Shell Redirection] dup2() successfully mirrored shell '>' and '<' operations.\n");
    return 0;
}
```

Output screenshot:

```
teja27@TEJA:~$ nano exp9_task4.c
teja27@TEJA:~$ gcc exp9_task4.c -o exp9_task4
teja27@TEJA:~$ ./exp9_task4
teja27@TEJA:~$ cat redirected_output.txt
[Shell Redirection] Restored stdout to terminal successfully.
[Shell Redirection] Successfully read from redirected stdin: 'Redirection Input Data From File
'[Shell Redirection] dup2() successfully mirrored shell '>' and '<' operations.
This output was redirected to redirected_output.txt using dup2()!
teja27@TEJA:~$
```